SIMATS ENGINEERING

ASSESSMENT TOOL 3: Modelling Comparative Analysis Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA0101**

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.*(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

Modelling Comparative Analysis Assignment

Code of Conduct:

I Shaik Mubarak(192425194) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	
10. Documentation, GitHub Repository and Presentation	2%	
Total	**100%**	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Question / Problem Statement
1	**Automobile Fault Diagnosis:** An automobile service center wants to identify vehicle faults based on symptoms such as engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. Model the problem using production rules, propositional logic, first-order logic, and Prolog. Demonstrate forward and backward chaining and compare the approaches based on expressiveness, inference, complexity, and suitability.
2	**Industrial Machine Fault Diagnosis:** A manufacturing plant needs to identify machine faults using observations such as abnormal vibration, high temperature, unusual noise, pressure variation, and reduced production speed. Develop different knowledge representation models and compare their ability to represent machine components, relationships, rules, and diagnostic conclusions.
3	**Cybersecurity Threat Detection:** A security operations center wants to identify threats from repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Model the threat-detection problem using production rules, propositional logic, first-order logic, and Prolog, and compare forward and backward reasoning for deriving security conclusions.
4	**Healthcare Diagnosis:** A healthcare system needs to identify possible diseases from symptoms such as fever, cough, fatigue, breathing difficulty, and body pain. Construct comparative knowledge models using production rules, propositional logic, first-order logic, and Prolog. Analyze which representation provides better expressiveness and reasoning capability for diagnosis.
5	**Agricultural Crop Disease Diagnosis:** Farmers need assistance in identifying crop diseases based on leaf color, spots, wilting, soil conditions, humidity, and pest observations. Represent the agricultural knowledge using different modelling approaches and compare their effectiveness in representing relationships, applying rules, and deriving recommendations.
6	**Banking Loan Eligibility:** A bank wants to determine loan eligibility using income, credit score, employment status, existing loans, and repayment history. Model the decision-making process using production rules, propositional logic, first-order logic, and Prolog, and compare the reasoning capabilities of each approach.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Problem Statement & Objectives	Describe the selected real-world problem, domain, decision-making requirement, and objectives of the modelling exercise.
2	Domain Knowledge & Facts	Identify entities, attributes, facts, relationships, conditions, and conclusions relevant to the selected problem.
3	Production Rule Model	Represent the domain using IF–THEN production rules and explain how the rules derive conclusions.
4	Propositional Logic Model	Represent the same problem using propositions, logical connectives, and suitable inference statements.
5	First-Order Logic Model	Represent entities, relationships, properties, and rules using predicates and quantifiers.
6	Prolog Model & Implementation	Convert the knowledge into Prolog facts, rules, predicates, and queries. Include relevant code and outputs.
7	Forward & Backward Chaining	Demonstrate both reasoning mechanisms using selected facts/goals and provide the inference sequence.
8	Comparative Analysis (Table structure given)	Compare the models based on expressiveness, inference mechanism, complexity, scalability, flexibility, explainability, and suitability.
9	Results, Discussion & Recommended Model	Analyze the results and justify which approach is most appropriate for the selected real-world problem. Discuss limitations and possible improvements.
10	Conclusion, References & GitHub	Summarize findings, provide references, and include the GitHub repository link containing Prolog code, models, test cases, and supporting materials.

Compulsory Comparative Table:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation				
Expressiveness				
Rule Representation				
Inference Mechanism				
Forward Chaining				
Backward Chaining				
Handling Relationships				
Scalability				
Explainability				
Ease of Implementation				
Suitability for the Problem				

TOPIC:4-HEALTHCARE DIAGNOSIS
HEALTHCARE DIAGNOSIS USING KNOWLEDGE REPRESENTATION MODELs

1. Problem Statement & Objectives

Problem Statement

A healthcare diagnosis system needs to identify possible diseases based on symptoms reported by a patient. Common symptoms considered in this system are:

- Fever
- Cough
- Fatigue
- Breathing difficulty
- Body pain

The same medical knowledge is represented using four different knowledge representation techniques:

1. Production Rules
2. Propositional Logic
3. First-Order Logic
4. Prolog

The models are then compared based on expressiveness, reasoning capability, complexity, scalability, flexibility, and explainability.

Note: This is an educational expert-system model, not a medical diagnostic tool.

Objectives

- Represent healthcare knowledge using different logical models.
- Identify possible diseases from given symptoms.
- Implement the knowledge using Prolog.
- Demonstrate forward and backward chaining.
- Compare the four representation techniques.
- Determine which model is most suitable for rule-based healthcare diagnosis.

2. Domain Knowledge & Facts

Entities

The main entity is the **Patient**.

Symptoms

The system considers:

- fever
- cough
- fatigue
- breathing_difficulty
- body_pain

Possible Diseases

The example system considers:

- Flu
- Common Cold
- Pneumonia
- COVID-like respiratory infection

Knowledge Table

Disease	Symptoms
Flu	Fever, cough, fatigue, body pain
Common Cold	Cough, fatigue
Pneumonia	Fever, cough, breathing difficulty
COVID-like infection	Fever, cough, fatigue, breathing difficulty

Example Patient Facts

Suppose patient p1 has:

Fever = Yes

Cough = Yes

Fatigue = Yes

Breathing Difficulty = Yes

Body Pain = No

From these facts, the system can identify possible diseases.

3. Production Rule Model

Production rules use the **IF–THEN** format.

Rules

Rule 1:

IF fever AND cough AND fatigue AND body pain

THEN possible flu.

Rule 2:

IF cough AND fatigue

THEN possible common cold.

Rule 3:

IF fever AND cough AND breathing difficulty

THEN possible pneumonia.

Rule 4:

IF fever AND cough AND fatigue AND breathing difficulty

THEN possible COVID-like infection.

Example

Given:

fever

cough

fatigue

breathing difficulty

Rule 3 is satisfied:

fever + cough + breathing difficulty

↓

possible pneumonia

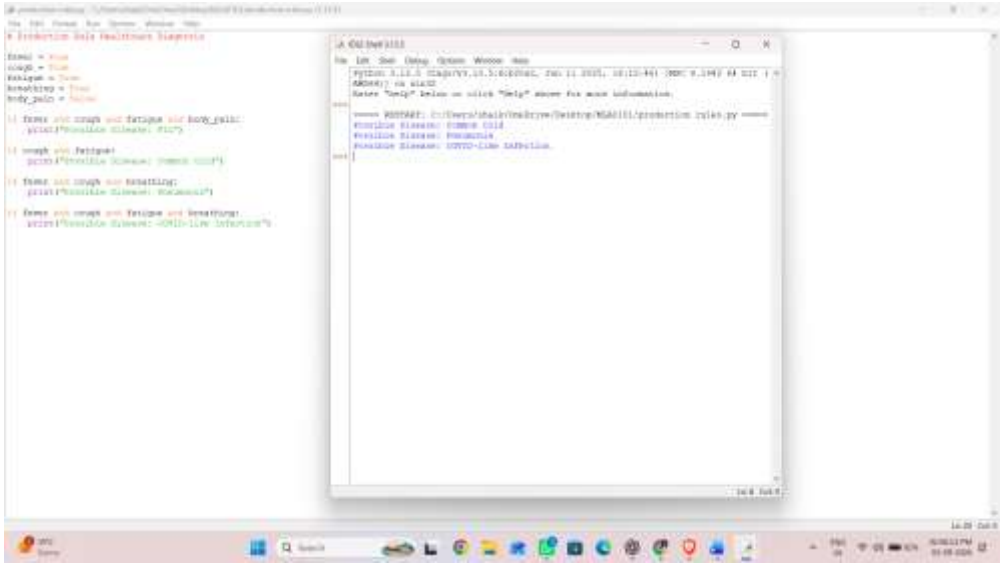
Rule 4 is also satisfied:

fever + cough + fatigue + breathing difficulty

↓

possible COVID-like infection

Therefore, the system produces multiple possible conclusions.



4. Propositional Logic Model

In propositional logic, each symptom is represented by a proposition.

Let:

F = Patient has fever

C = Patient has cough

T = Patient has fatigue

B = Patient has breathing difficulty

P = Patient has body pain

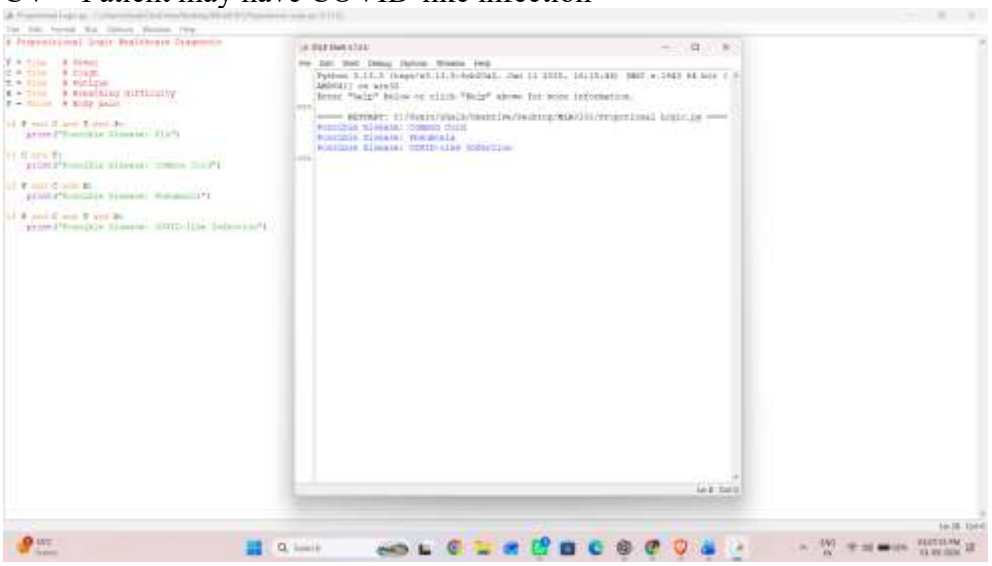
Let the disease propositions be:

FL = Patient may have flu

CC = Patient may have common cold

PN = Patient may have pneumonia

CV = Patient may have COVID-like infection



Rules

Flu

$F \land C \land T \land P \rightarrow FL$

Meaning:
If the patient has fever, cough, fatigue, and body pain, then flu is possible.

Common Cold

$C \land T \rightarrow CC$

Pneumonia

$F \land C \land B \rightarrow PN$

COVID-like Infection

$F \land C \land T \land B \rightarrow CV$

Example Inference

Given:

F = True
C = True
T = True
B = True
P = False

We can derive:

$F \land C \land B \rightarrow PN$

Therefore:

PN = True

Also:

$F \land C \land T \land B \rightarrow CV$

Therefore:

CV = True

5. First-Order Logic Model

First-Order Logic is more expressive because it allows us to represent objects, properties, relationships, variables, and quantifiers.

Predicates

Fever(x)
Cough(x)
Fatigue(x)
BreathingDifficulty(x)
BodyPain(x)
Flu(x)
CommonCold(x)
Pneumonia(x)
CovidLike(x)

Here x represents a patient.

Rules

Flu

$\forall x [(Fever(x) \land Cough(x) \land Fatigue(x) \land BodyPain(x)) \rightarrow Flu(x)]$

Common Cold

$\forall x [(Cough(x) \land Fatigue(x)) \rightarrow CommonCold(x)]$

Pneumonia

$\forall x [(Fever(x) \land Cough(x) \land BreathingDifficulty(x)) \rightarrow Pneumonia(x)]$

COVID-like Infection

$\forall x [(Fever(x) \land Cough(x) \land Fatigue(x)$

∧ BreathingDifficulty(x))
→ CovidLike(x)]

Patient Facts

Fever(p1)
Cough(p1)
Fatigue(p1)
BreathingDifficulty(p1)
Using the pneumonia rule:
Fever(p1) ∧ Cough(p1) ∧ BreathingDifficulty(p1)
Therefore:
Pneumonia(p1)

6. Prolog Model & Implementation

Prolog Program

```
% Symptoms of patient p1
fever(p1).
cough(p1).
fatigue(p1).
breathing_difficulty(p1).
% Diagnosis rules
disease(Patient, flu) :-
   fever(Patient),
   cough(Patient),
   fatigue(Patient),
   body_pain(Patient).
disease(Patient, common_cold) :-
   cough(Patient),
   fatigue(Patient).
disease(Patient, pneumonia) :-
   fever(Patient),
   cough(Patient),
   breathing_difficulty(Patient).
disease(Patient, covid_like) :-
   fever(Patient),
   cough(Patient),
   fatigue(Patient),
   breathing_difficulty(Patient).
```

Query 1

?- disease(p1, Disease).

Output

Disease = common_cold ;
Disease = pneumonia ;
Disease = covid_like.
The exact order can vary depending on the Prolog implementation/rules.

Query 2

?- disease(p1, pneumonia).

Output

true.

Query 3

?- disease(p1, flu).

Output
false.
Because body_pain(p1) was not provided.
7. Forward & Backward Chaining
A. Forward Chaining
Forward chaining starts with **known facts** and applies rules to derive conclusions.
Initial Facts
Fever(p1)
Cough(p1)
Fatigue(p1)
BreathingDifficulty(p1)
Step 1
Check:
Fever + Cough + BreathingDifficulty
Matches pneumonia rule.
Therefore:
Pneumonia(p1)
Step 2
Check:
Fever + Cough + Fatigue + BreathingDifficulty
Matches COVID-like infection rule.
Therefore:
CovidLike(p1)
Forward Chaining Flow

Facts
↓
Check Rules
↓
Matching Conditions
↓
New Conclusions

B. Backward Chaining
Backward chaining starts with a **goal** and works backward to determine whether the required facts are available.
Goal
Pneumonia(p1)
The system searches for a rule that can produce pneumonia.
Fever(X) ∧ Cough(X) ∧ BreathingDifficulty(X)
→ Pneumonia(X)
Now it checks:
Fever(p1) ✓
Cough(p1) ✓
BreathingDifficulty(p1) ✓
All conditions are true.
Therefore:
Pneumonia(p1) = True
Backward Chaining Flow

Goal: Pneumonia(p1)
↓

Find matching rule

↓

Check required conditions

↓

All facts available

↓

Goal satisfied

Prolog naturally supports backward reasoning through its query-driven execution.

8. Comparative Analysis

Criteria	**Production Rules**	**Propositional Logic**	**First-Order Logic**	**Prolog**
Expressiveness	Medium	Low	Very High	High
Representation	IF-THEN rules	True/False propositions	Predicates and quantifiers	Facts and rules
Variables	Limited	No	Yes	Yes
Quantifiers	No	No	Yes	Implicit through variables
Inference	Forward/Rule based	Logical inference	Logical inference	Mainly backward chaining
Complexity	Low	Medium	High	Medium
Scalability	Medium	Low	High	High
Flexibility	Medium	Low	Very High	High
Explainability	High	High	Medium	High
Implementation	Easy	Easy	Difficult	Easy for rule systems
Suitability for diagnosis	Good	Limited	Excellent	Excellent
Handling relationships	Limited	Poor	Excellent	Excellent
Practical implementation	Good	Limited	Complex	Very good

Overall Comparison

Production Rules

- Very easy to understand.
- Good for simple expert systems.
- Easy to explain to users.
- Can become difficult to manage when the number of rules increases.

Propositional Logic

- Simple and mathematically clear.
- Suitable for simple Boolean reasoning.
- Does not naturally represent variables or relationships.
- Less suitable for a larger healthcare knowledge base.

First-Order Logic

- Most expressive representation.
- Supports variables, predicates, relationships, and quantifiers.
- Can represent complex healthcare knowledge.
- More difficult to implement and reason with directly.

Prolog

- Designed for symbolic and logical reasoning.
- Uses facts and rules naturally.
- Supports variables and recursive rules.
- Queries make diagnosis easy to test.
- Particularly suitable for small-to-medium rule-based expert systems.

9. Results, Discussion & Recommended Model

Results

The same healthcare diagnosis problem was represented using four different techniques.

For patient p1:

Fever = Yes

Cough = Yes

Fatigue = Yes

Breathing Difficulty = Yes

Body Pain = No

The system can derive:

Possible Common Cold

Possible Pneumonia

Possible COVID-like Infection

Flu is not derived because body pain is absent.

Discussion

Each representation has different strengths.

Production rules are easy to understand but become difficult to maintain when the number of rules increases.

Propositional logic provides a simple mathematical representation but cannot easily represent individual patients, variables, and relationships.

First-Order Logic provides the greatest expressive power because it can represent objects, properties, relationships, variables, and quantifiers.

Prolog provides a practical implementation of logical reasoning. Facts and rules can be directly converted into executable knowledge, and queries can be used to perform diagnosis.

Recommended Model

Prolog is recommended for this project.
Reasons:

1. Simple fact and rule representation.
2. Supports variables.
3. Provides logical inference.
4. Supports backward chaining.
5. Easy to modify and extend.
6. Suitable for expert-system development.
7. Queries provide direct diagnostic results.
8. Better practical balance between expressiveness and implementation complexity.

However, First-Order Logic has greater theoretical expressiveness. Therefore:
FOL is the most expressive model, while Prolog is the most practical model for implementing this rule-based healthcare diagnosis system.

Limitations

- The system uses only a small number of symptoms.
- Symptoms can occur in multiple diseases.
- Real medical diagnosis requires medical history, examination, tests, and professional judgment.
- The system does not calculate disease probabilities.
- It does not handle uncertainty effectively.

Future Improvements
The system can be improved by adding:

- More diseases and symptoms.
- Disease severity levels.
- Patient age and medical history.
- Laboratory-test results.
- Probabilistic reasoning.
- Confidence scores.
- Natural-language symptom input.
- A graphical/web interface.

10. Conclusion & References

Conclusion
This project demonstrated four knowledge representation techniques for a healthcare diagnosis problem: Production Rules, Propositional Logic, First-Order Logic, and Prolog.Production rules provide a simple and understandable representation. Propositional logic is useful for basic Boolean reasoning but has limited expressiveness. First-Order Logic provides the highest expressive power because it supports variables, predicates, relationships, and quantifiers.

Prolog provides a practical way to implement the knowledge base and perform logical reasoning using facts, rules, and queries.Therefore, First-Order Logic is the most expressive representation, while Prolog is recommended as the practical implementation model for the proposed healthcare diagnosis expert system.

References

1. Stuart Russell and Peter Norvig, *Artificial Intelligence: A Modern Approach*.
2. Ivan Bratko, *Prolog Programming for Artificial Intelligence*.
3. George F. Luger, *Artificial Intelligence: Structures and Strategies for Complex Problem Solving*.
4. Documentation and learning resources on Prolog and knowledge representation.

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear or poorly understood.
2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8% – Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.
3. Production Rule Modelling	15%	13–15% – Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12% – Rules are appropriate with minor errors or omissions.	7–9% – Basic rules are developed but several are incomplete or unclear.	0–6% – Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8% – Good propositional representation with minor errors.	5–6% – Basic representation with limited logical formulation.	0–4% – Representation is incorrect or incomplete.
5. First-Order Logic Modelling	15%	13–15% – Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL statements.	10–12% – Good FOL model with minor errors or omissions.	7–9% – Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6% – FOL model is largely incorrect or missing.

6. Prolog Modelling and Implementation	15%	13–15% – Successfully converts the knowledge model into Prolog facts, rules, predicates, and queries with correct outputs.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works but has limited functionality.	0–6% – Implementation is incomplete or non-functional.
7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8% – Both mechanisms are demonstrated with minor gaps.	5–6% – Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10% – Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8% – Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4% – Little or no meaningful comparison.
9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2% – Provides good analysis and reasonable justification.	1% – Basic discussion with limited justification.	0% – No meaningful analysis or justification.
10. Documentation, GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5% – Good documentation and presentation with minor issues.	1% – Basic documentation with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				